



IMD1116

COMPUTAÇÃO DE ALTO DESEMPENHO

MPI:
COMUNICAÇÃO PONTO A PONTO
MODOS DE COMUNICAÇÃO

Prof. Samuel Xavier de Souza
Prof. Carla Santana

Método do Trapézio - MPI

Atv Extra - Implemente, uma **versão paralela em MPI** de um programa para o cálculo de integrais definidas por meio do método do trapézio. O programa deve permitir calcular a integral de uma função $f(x)$ em um intervalo $[a,b]$, utilizando um número n de subdivisões.

O processo 0 deverá receber as áreas parciais calculadas por cada processo. O código deve ser implementado de forma a permitir execução com **mais de dois processos**, utilizando apenas comunicações ponto a ponto.

Você terá 15-20 minutos para implementar o programa.



PROGRAMAÇÃO PARALELA

- Objetivos deste tópico
 - MPI
 - Comunicação ponto a ponto
 - síncrono e assíncrono
 - bloqueante e não bloqueante
 - Compreender os conceitos básicos de comunicação ponto-a-ponto em MPI e aprender a utilizar as funções principais

Programação em Memória Compartilhada

Comunicação entre as tarefas é realizada por meio de variáveis **armazenadas em um espaço de memória compartilhado**.



Uma API para programação paralela em memória compartilhada.

Open Multi-Processing

www.openmp.org/



The screenshot shows the OpenMP website homepage. At the top is the OpenMP logo with the tagline "The OpenMP API specification for parallel programming". Below the logo is a navigation bar with links for Home, Specifications, Community, Resources, News & Events, and About. The main content area features a large teal banner with the heading "New OpenMP API 6.0 Examples Document" and a subheading "Provides programming examples of the OpenMP API, including new features found in the OpenMP 6.0 Specification." A blue "DOWNLOAD" button is positioned below the text. The bottom section, titled "Latest News and Events", includes social media icons and three news items: "OpenMP python" with a logo, "OpenMP ARB announces new Board Members van der Pas and Stotzer", and "Download the free OpenMP Examples book..." with a book cover image.

OpenMP
The OpenMP API specification for parallel programming

Home Specifications Community Resources News & Events About

New OpenMP API 6.0 Examples Document

Provides programming examples of the OpenMP API, including new features found in the OpenMP 6.0 Specification.

DOWNLOAD

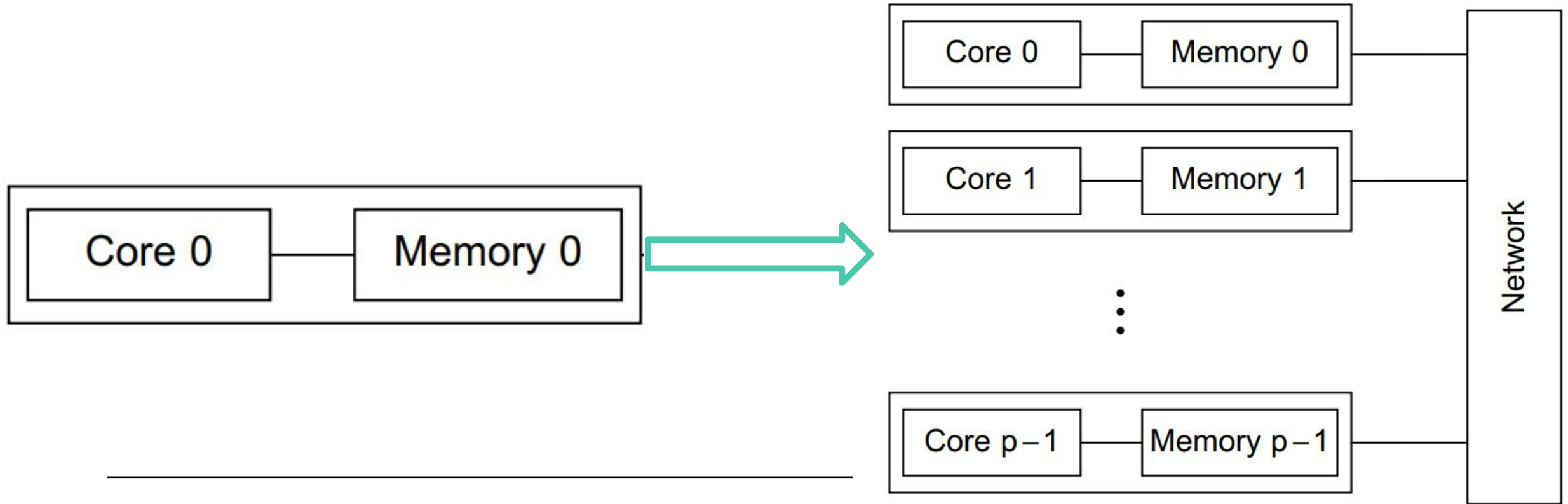
Latest News and Events

OpenMP python

OpenMP ARB announces new Board Members van der Pas and Stotzer

Download the free OpenMP Examples book...

Seu problema não cabe mais em um único computador.

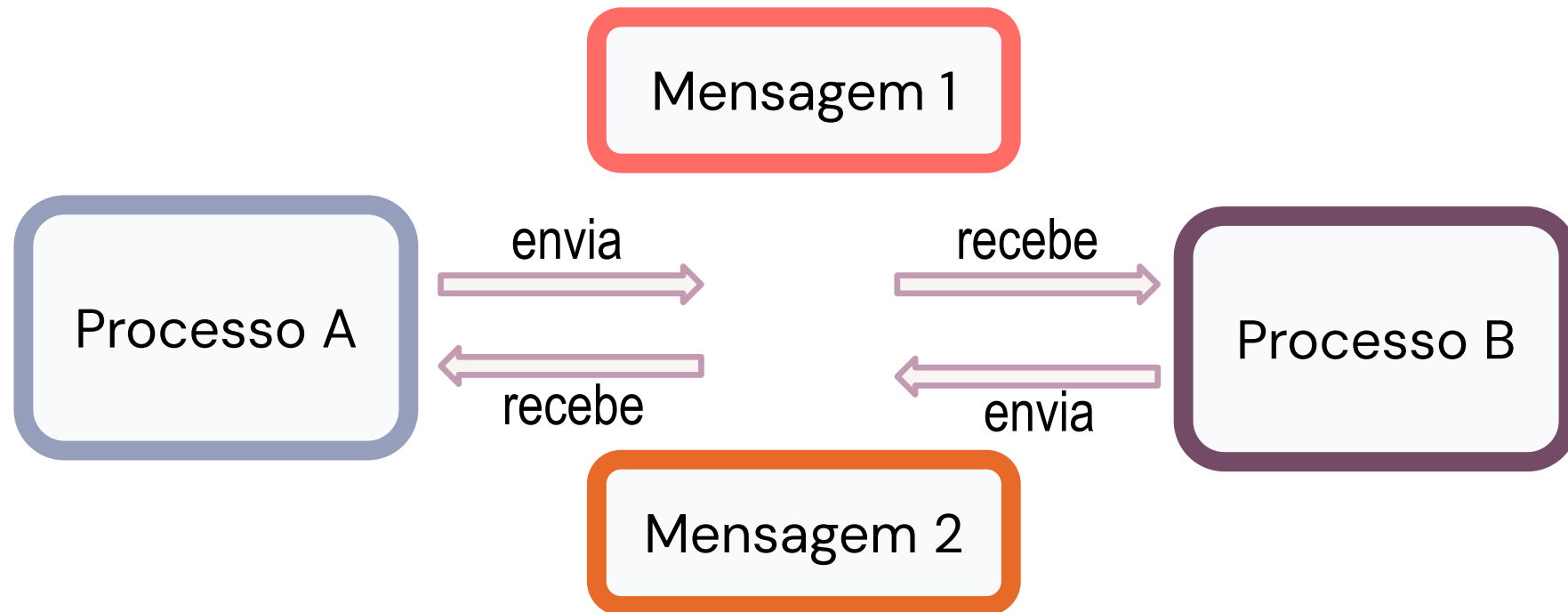


Programação em Memória Distribuída com MPI



Programação em Memória Distribuída

Programação por Troca de mensagens



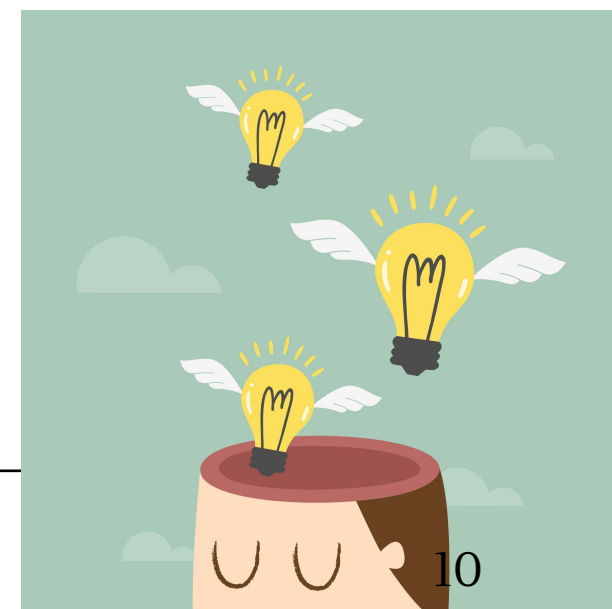
Síncrono X Assíncrono

- Síncrono
 - Há sincronização entre emissor e receptor
 - O envio só acontece quando o receptor está pronto
 - Existe um “encontro” entre os processos
- Assíncrono
 - Não há necessidade de sincronização imediata
 - O emissor envia e segue executando
 - O receptor pode receber depois

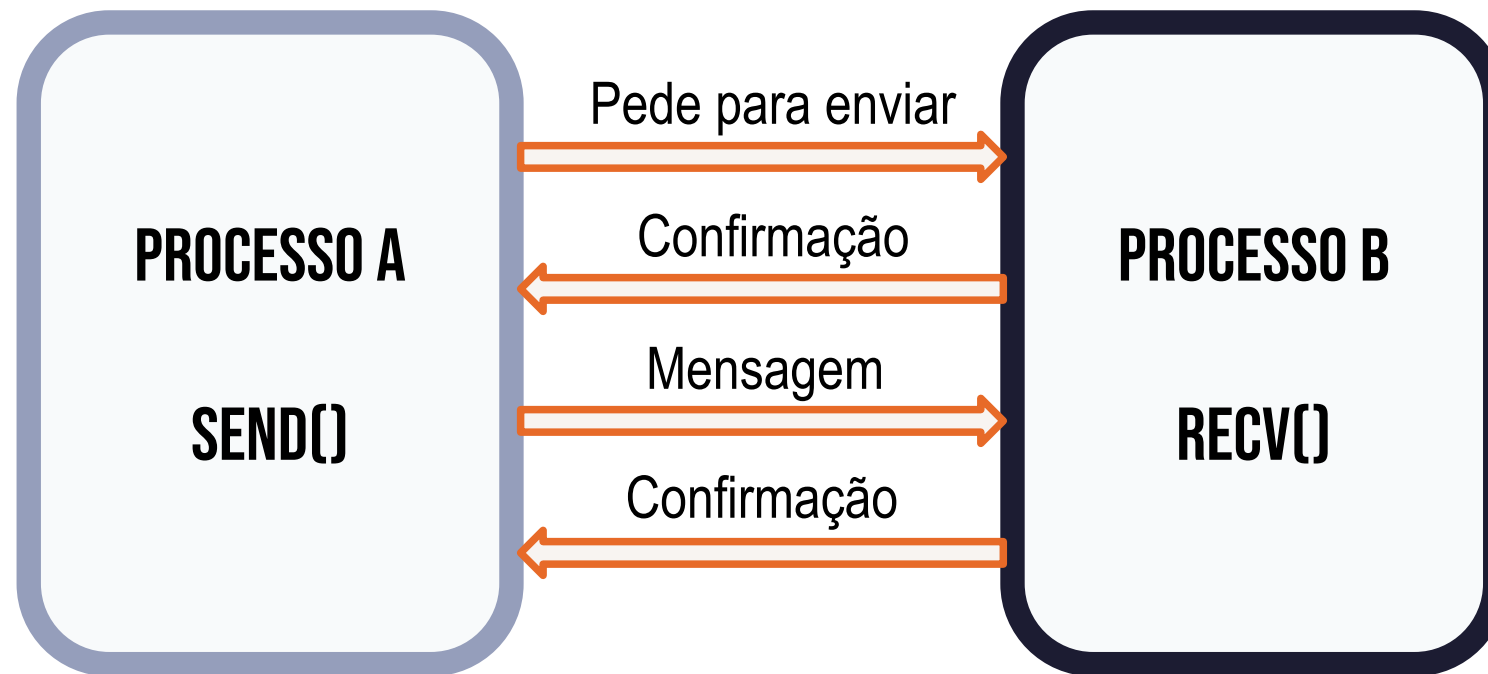


Bloqueante X Não Bloqueante

- Bloqueante
 - A função só retorna quando a operação termina
 - O processo fica “parado” esperando
- Não bloqueante
 - A função retorna imediatamente
 - A operação continua em segundo plano

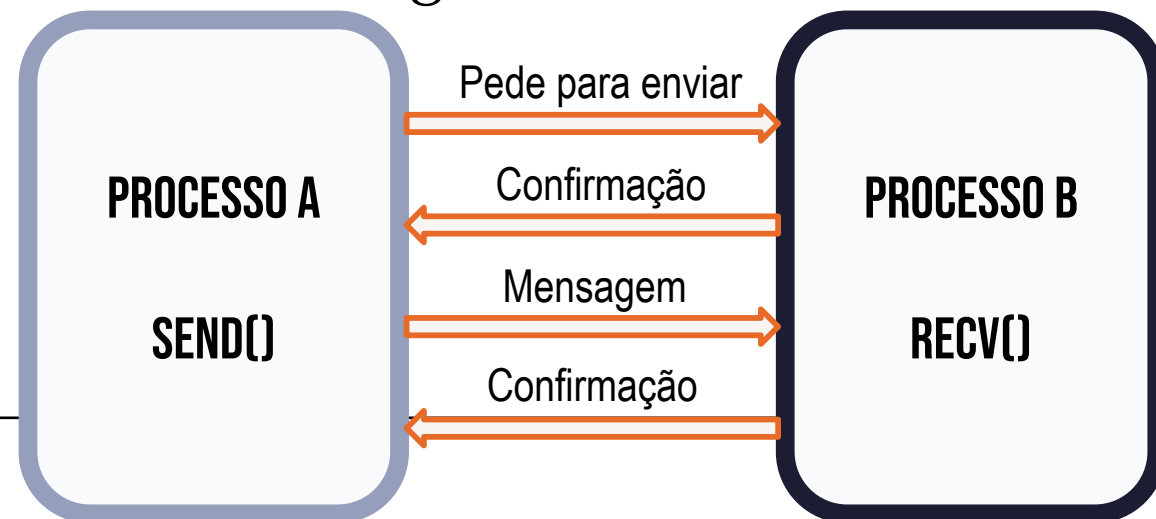


Modo de comunicação síncrona

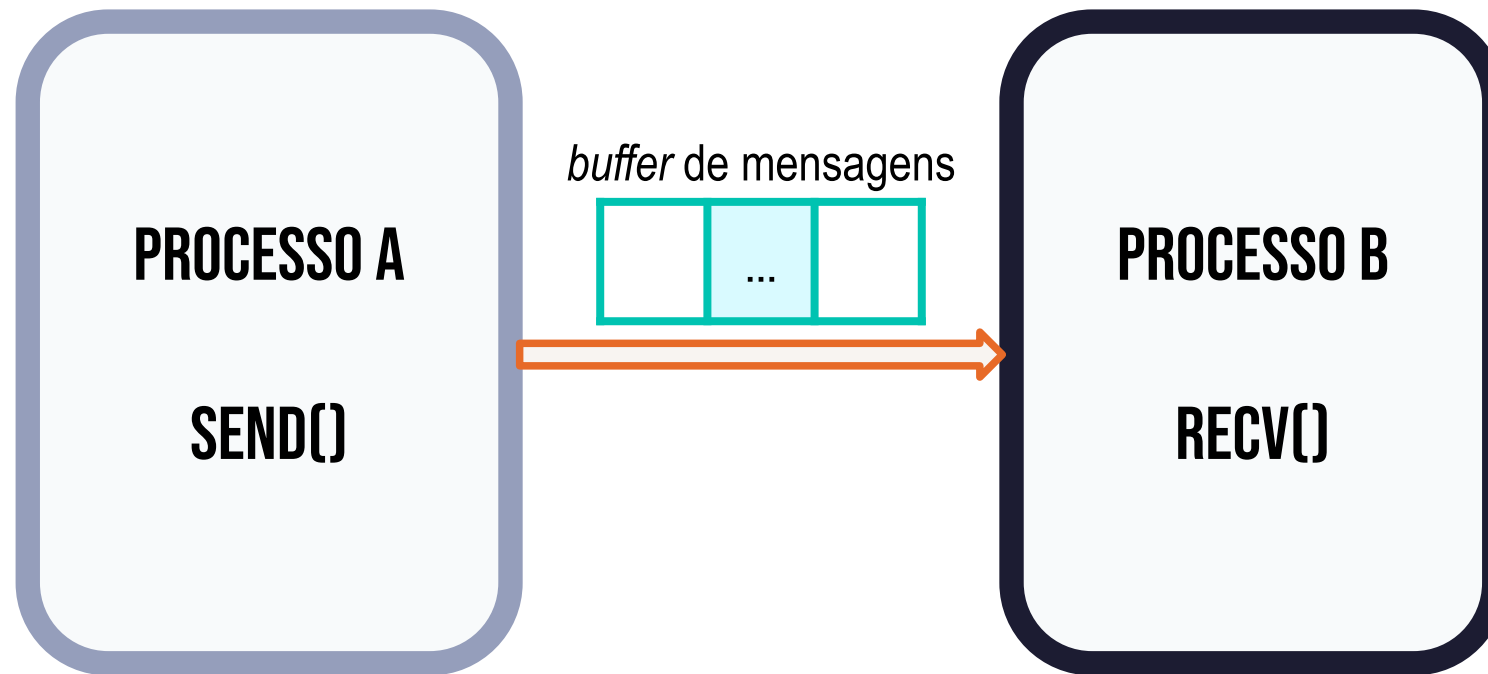


Modo de comunicação síncrona

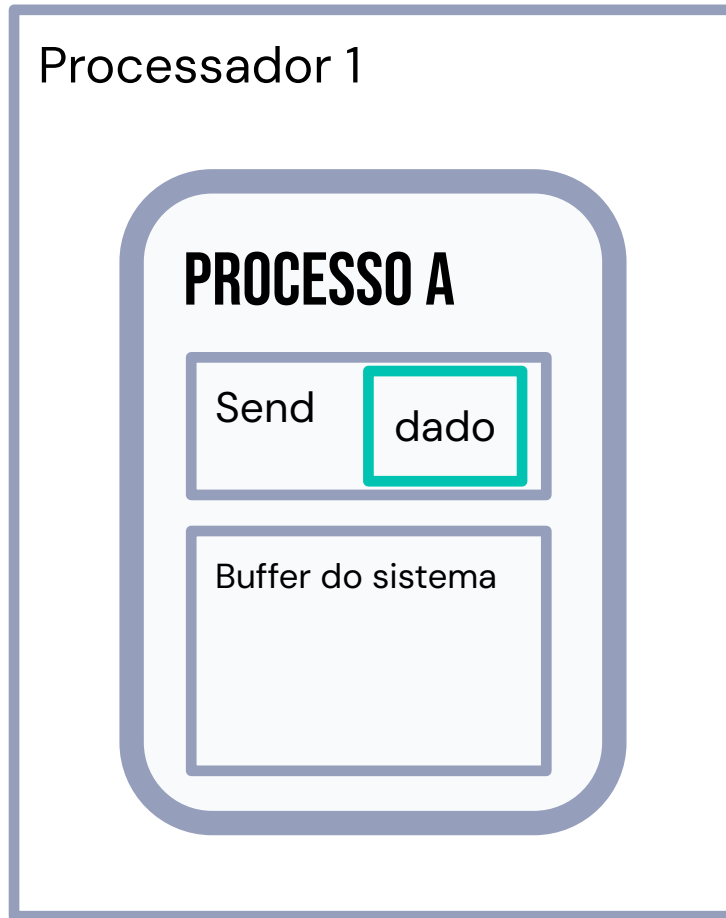
- No modo de comunicação síncrona, tanto o envio como a recepção de mensagens são **bloqueantes** .
- Simples e seguro e não requer uso de *buffers*
- **Sem possibilidade de haver superposição** entre o processamento da aplicação e a transmissão das mensagens



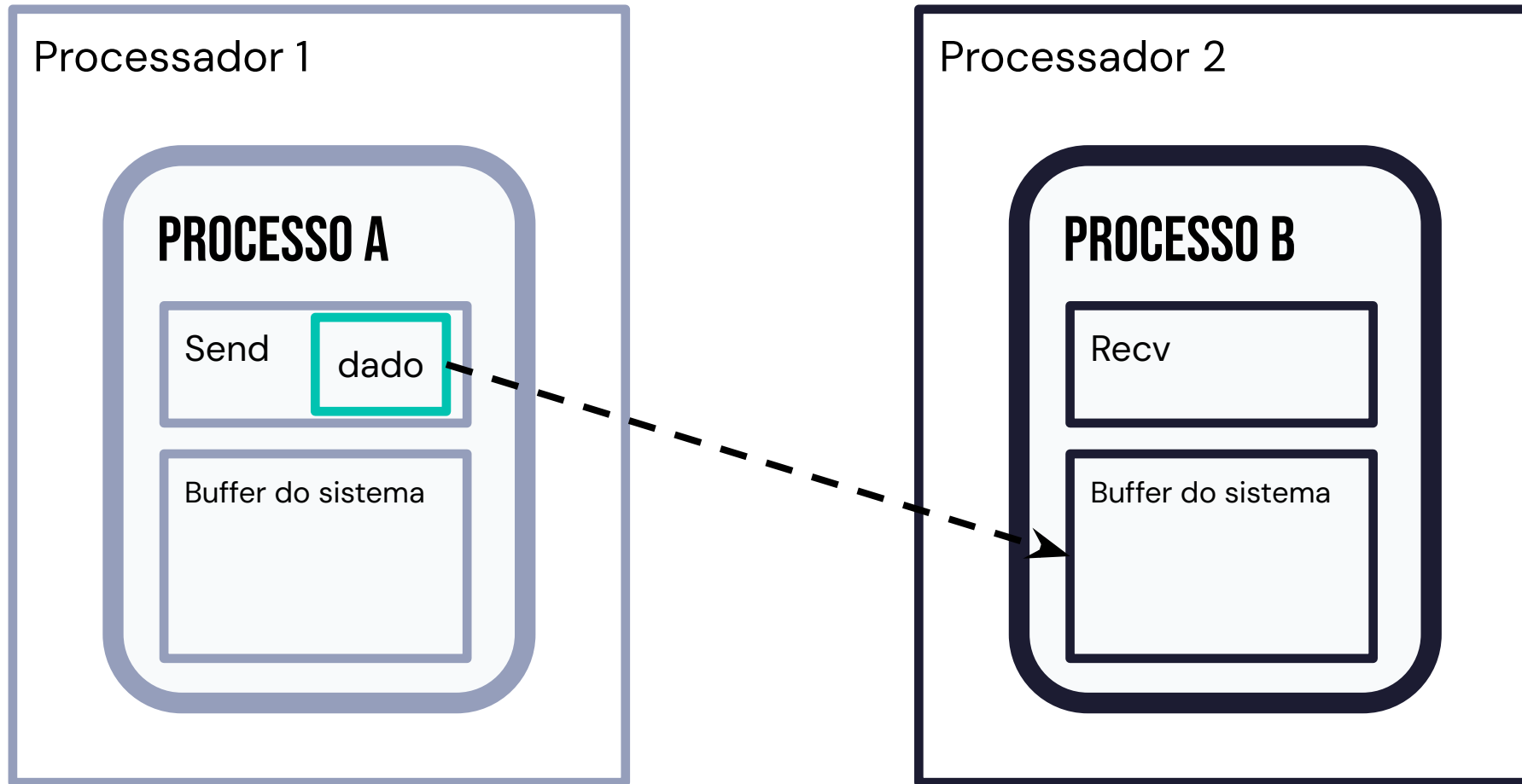
Modo de comunicação assíncrona



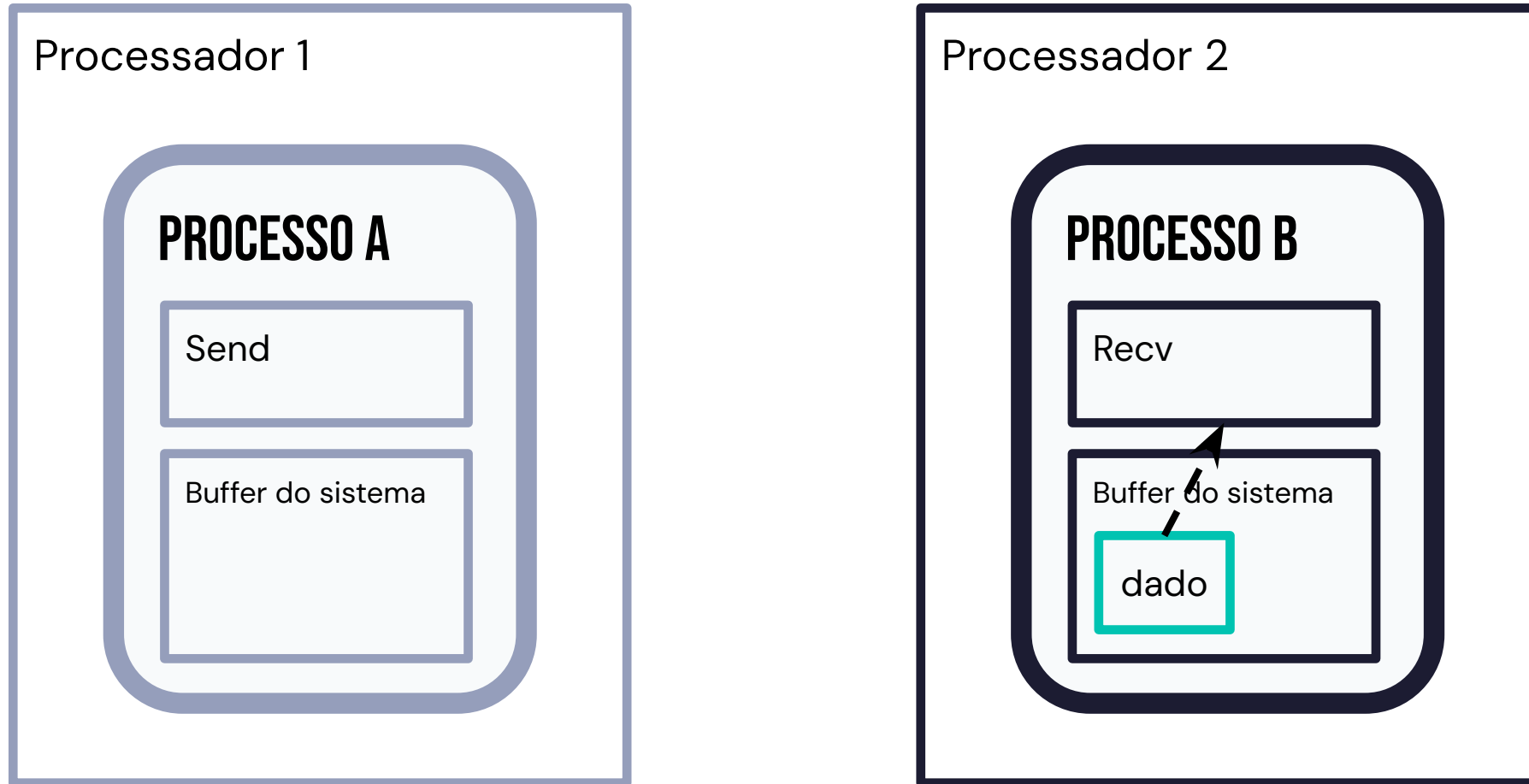
Buffering



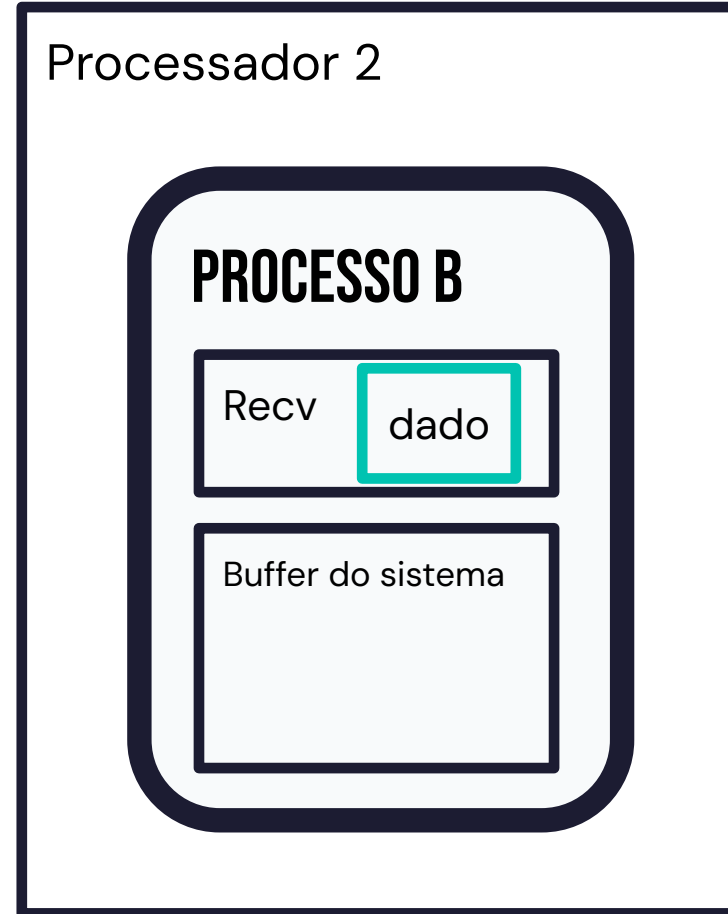
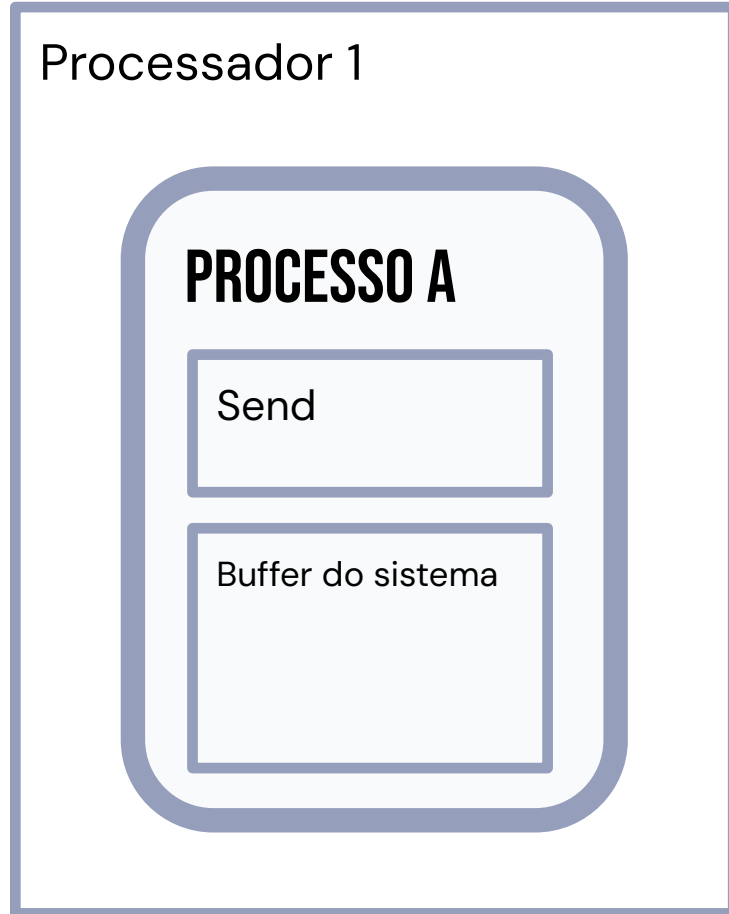
Buffering



Buffering

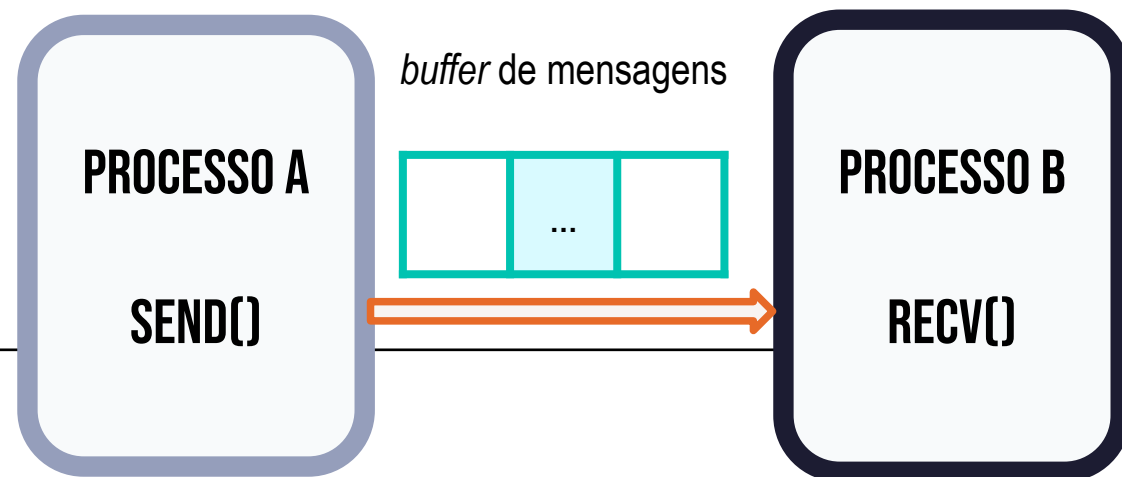


Buffering



Modo de comunicação assíncrona

- Operação de envio (*send*) se completa a partir da cópia de todo o conteúdo da mensagem do espaço de endereçamento do usuário para uma área correspondente (*buffer*) da biblioteca ou do sistema operacional.
- Permite **superposição no tempo entre o processamento da aplicação e a transmissão das mensagens**, aumentando as possibilidades de exploração de paralelismo.
- Pode ser bloqueante ou não bloqueante



Message Passing Interface

Padrão de programação paralela baseado em troca de mensagens entre processos. Ele **define um conjunto de funções (API)** que permitem que programas rodem em múltiplos processadores ou máquinas, comunicando-se explicitamente. **Não é uma biblioteca, é a especificação de como a biblioteca deve ser.**



www.mpi-forum.org/

MPI Forum

DOCS CURRENT MPI EFFORTS ▼ MEETINGS ▼ RESOURCES ▼

MPI Forum

This website contains information about the activities of the MPI Forum, which is the standardization forum for the Message Passing Interface (MPI). You may find standard documents, information about the activities of the MPI forum, and links to comment on the MPI Document using the navigation at the top of the page.

[Link to the central MPI-Forum GitHub Presence](#)

Current Efforts

The MPI Forum is currently pursuing at least one future version of the MPI Standard:

- [MPI Next](#): A standard update following MPI 5.0 with currently undefined features

More details can be found in the respective sections.

Implementações Open Source

MPICH

mpich.org

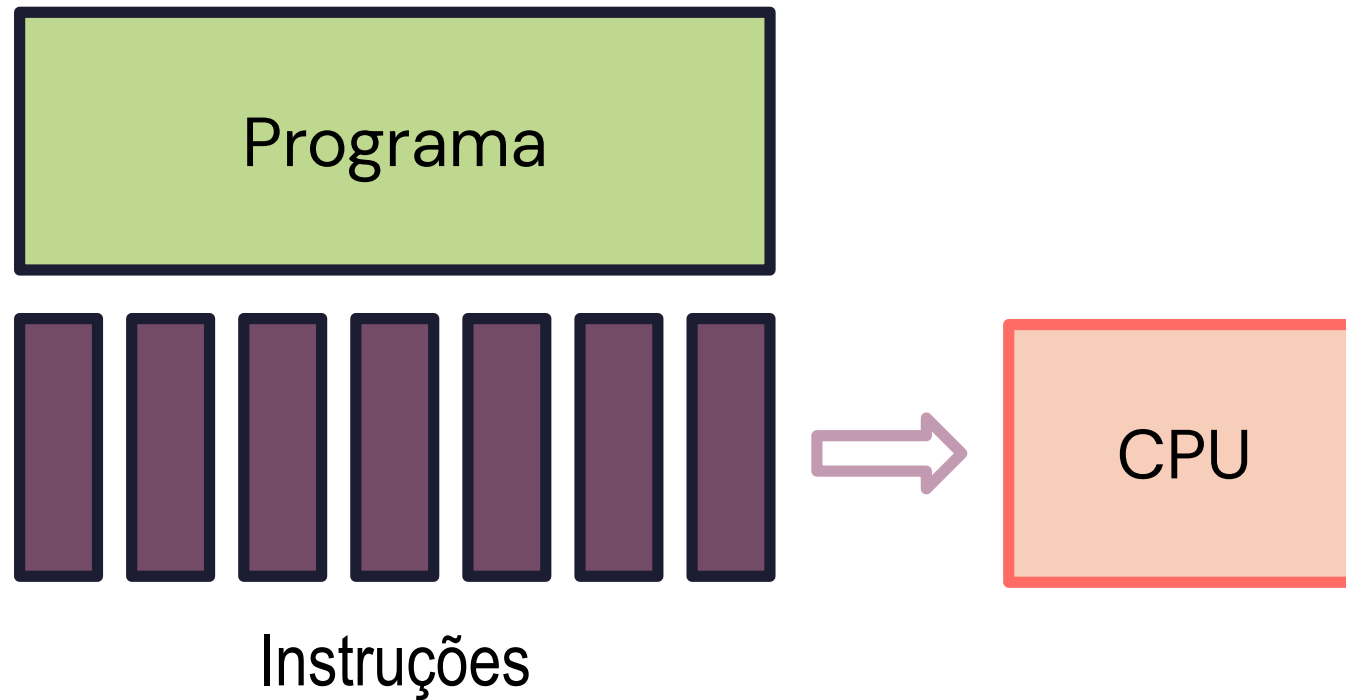
OpenMPI

open-mpi.org

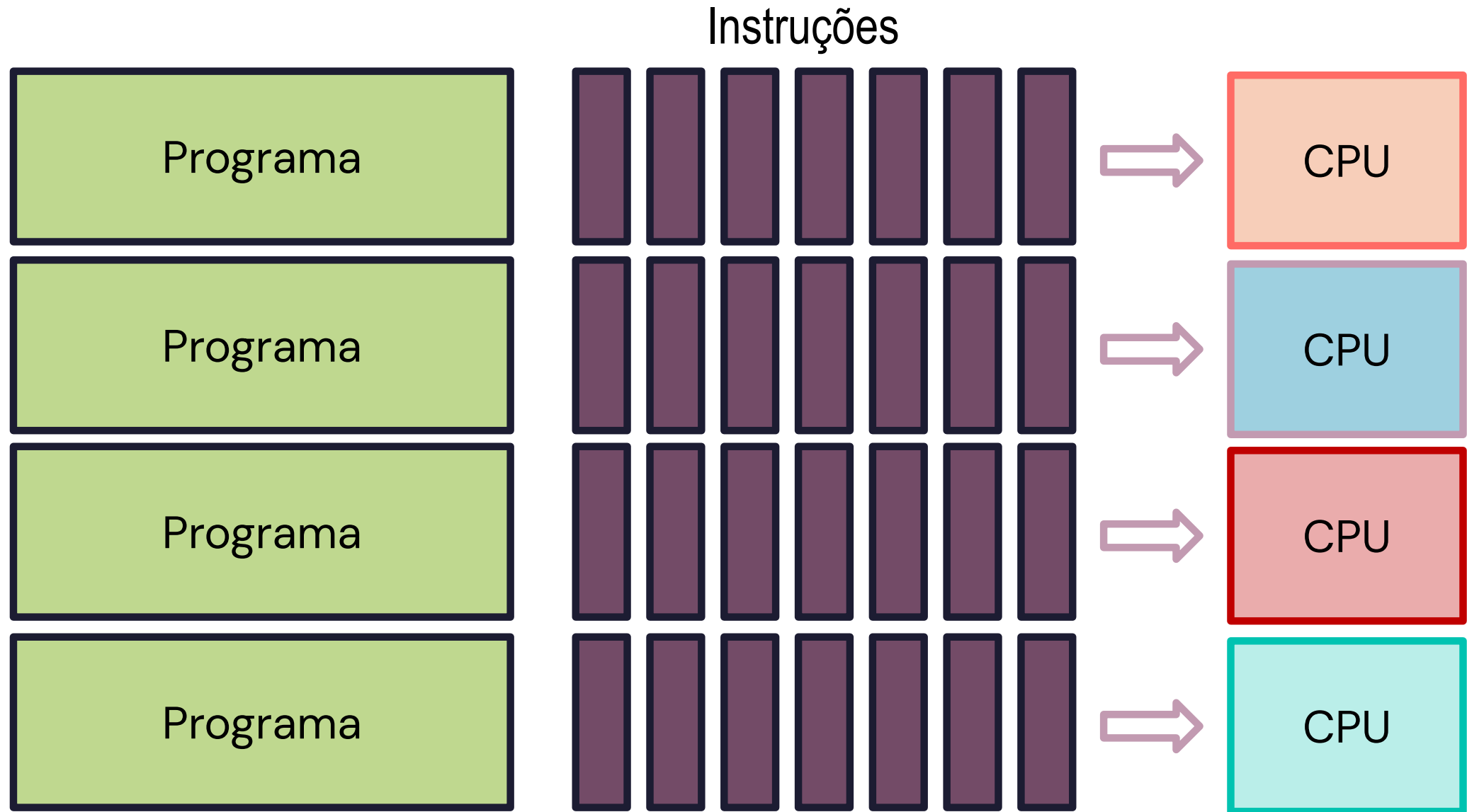
MPI segue o modelo de Programação SPMD (Single Program Multiple Data)

- O **processador gerente** envia e gerencia um “**mesmo programa**” em todas as máquinas do sistema distribuído
- **Cada parte do programa quebrado é chamada de processo.**
- Os processos podem ser executados **em uma única máquina ou em várias máquinas.**

Como funciona o MPI?

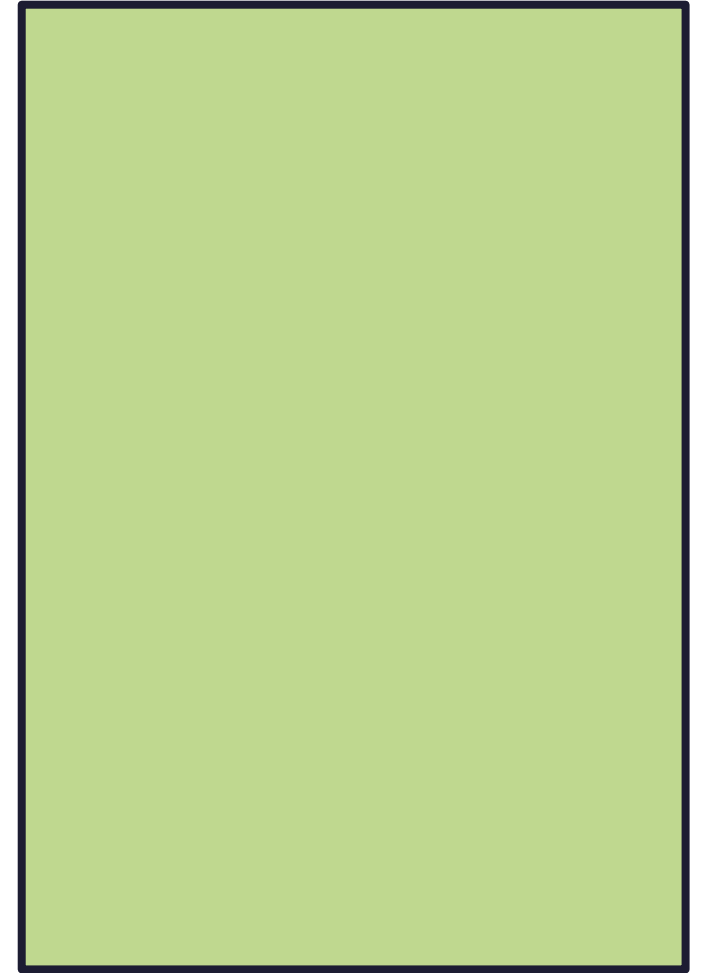


Como funciona o MPI?



MPI

Região de código sequencial



Região de código sequencial

MPI

Região de código sequencial

Configuração
Inicializações ...

Região de código sequencial

MPI

Região de código sequencial

Configuração
Inicializações ...
MPI_Init()

Região de código sequencial

MPI

Região de código sequencial

Configuração
Inicializações ...

MPI_Init()

Região de código paralelo

MPI

Região de código sequencial

Configuração
Inicializações ...

MPI_Init()

Região de código paralelo

MPI_Finalize()

Fim do Programa

Região de código sequencial

MPI

```
# include "mpi.h"

int main ( int argc , char * argv []) {
    /* Seção de código sequencial */
    /* Nenhuma função MPI pode ser chamada antes deste ponto */
    MPI_Init (& argc , & argv ) ;
    /* Seção de código paralelo */
    MPI_Finalize () ;
    /* Nenhuma função MPI pode ser chamada depois deste ponto */
    /* Seção de código sequencial */
    return (0) ;
}
```

Região de código sequencial

Configuração
Inicializações ...

MPI_Init()

Região de código paralelo

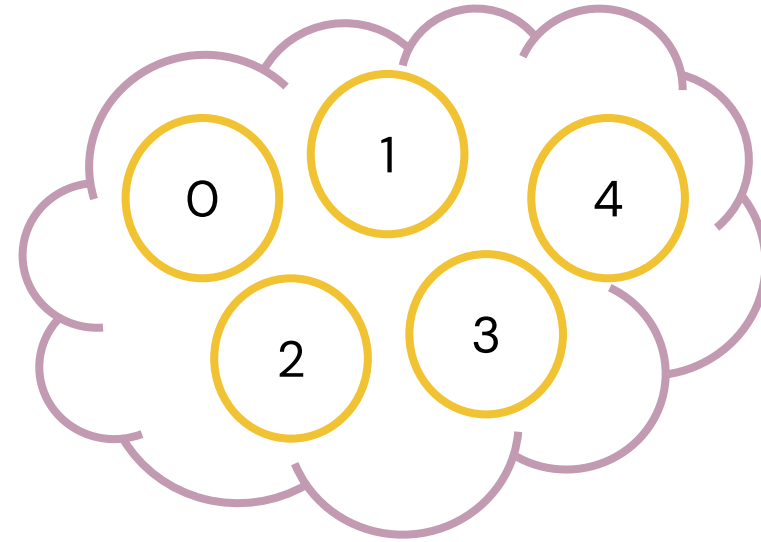
MPI_Finalize()

Fim do Programa

Região de código sequencial

Hello World

Comunicadores : Define o universo de processos envolvidos em uma operação de comunicação (**MPI_COMM_WORLD**)



- **Grupo:** conjunto ordenado de processos contidos no comunicador, cuja ordem é chamada de rank.
- **Contexto:** um rótulo (número inteiro único) definido pelo sistema, diferenciando um comunicador do outro.

Hello World

Comunicadores : Define o universo de processos envolvidos em uma operação de comunicação (**MPI_COMM_WORLD**)

Rank: Número que identifica o processo para aquele grupo de comunicação

Size: Quantidade de processos envolvidos na comunicação

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    int rank, size;

    // Inicializa o ambiente MPI
    MPI_Init(&argc, &argv);

    // Obtém o identificador do processo
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Obtém o número total de processos
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Cada processo imprime sua mensagem
    printf("Hello World do processo %d de %d\n", rank, size);

    // Finaliza o ambiente MPI
    MPI_Finalize();

    return 0;
}
```

Hello World

Comunicadores : Define o universo de processos envolvidos em uma operação de comunicação (**MPI_COMM_WORLD**)

Rank: Número que identifica o processo para aquele grupo de comunicação

Size: Quantidade de processos envolvidos na comunicação

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    int rank, size;

    // Inicializa o ambiente MPI
    MPI_Init(&argc, &argv);

    // Obtém o identificador do processo
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Obtém o número total de processos
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Cada processo imprime sua mensagem
    printf("Hello World do processo %d de %d\n", rank, size);

    // Finaliza o ambiente MPI
    MPI_Finalize();

    return 0;
}
```

Hello World

Comunicadores : Define o universo de processos envolvidos em uma operação de comunicação (**MPI_COMM_WORLD**)

Rank: Número que identifica o processo para aquele grupo de comunicação

Size: Quantidade de processos envolvidos na comunicação

```
#include <mpi.h>
#include <stdio.h>
int main(int argc, char *argv[]) {
    int rank, size;

    // Inicializa o ambiente MPI
    MPI_Init(&argc, &argv);

    // Obtém o identificador do processo
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    // Obtém o número total de processos
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    // Cada processo imprime sua mensagem
    printf("Hello World do processo %d de %d\n", rank, size);

    // Finaliza o ambiente MPI
    MPI_Finalize();

    return 0;
}
```

Compilar o código e executar

Compilar

```
mpicc meu_programa_omp.c -o programa
```

Executar

```
mpirun -np 4 ./programa
```

Comunicação Ponto-a-ponto

```
int main(int argc, char *argv[]) {
    int rank, size;
    char mensagem[100];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0)
            printf("Execute com pelo menos 2 processos!\n");
        MPI_Finalize();
        return 0;
    }
    if (rank == 0) {
        // Processo 0 envia mensagem
        strcpy(mensagem, "Olá do processo 0!");
        MPI_Send(mensagem, strlen(mensagem) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou mensagem.\n");
    } else if (rank == 1) {
        // Processo 1 recebe mensagem
        MPI_Recv(mensagem, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu: %s\n", mensagem);
    }
    MPI_Finalize();
    return 0;
}
```


Comunicação Ponto-a-ponto

- **MPI_Send** : envia um mensagem
- Rotina de envio só retorna depois que todos os dados forem copiados das variáveis utilizadas para o envio da mensagem.
- Mensagens maiores que o buffer são enviadas no modo síncrono (bloqueante)

```
int main(int argc, char *argv[]) {
    int rank, size;
    char mensagem[100];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0)
            printf("Execute com pelo menos 2 processos!\n");
        MPI_Finalize();
        return 0;
    }
    if (rank == 0) {
        // Processo 0 envia mensagem
        strcpy(mensagem, "Olá do processo 0!");
        MPI_Send(mensagem, strlen(mensagem) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou mensagem.\n");
    } else if (rank == 1) {
        // Processo 1 recebe mensagem
        MPI_Recv(mensagem, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu: %s\n", mensagem);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto

- **MPI_Send** : envia um mensagem
- Rotina de envio só retorna depois que todos os dados forem copiados das variáveis utilizadas para o envio da mensagem.
- Mensagens maiores que o buffer são enviadas no modo síncrono (bloqueante)
- **MPI_Recv** : recebe uma mensagem.
- Rotina de recepção só retorna quando os dados recebidos estiverem disponíveis nas variáveis utilizadas nas rotinas de recepção.
- MPI_Recv é bloqueante

```
int main(int argc, char *argv[]) {
    int rank, size;
    char mensagem[100];
    MPI_Init(&argc, &argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    if (size < 2) {
        if (rank == 0)
            printf("Execute com pelo menos 2 processos!\n");
        MPI_Finalize();
        return 0;
    }
    if (rank == 0) {
        // Processo 0 envia mensagem
        strcpy(mensagem, "Olá do processo 0!");
        MPI_Send(mensagem, strlen(mensagem) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD);
        printf("Processo 0 enviou mensagem.\n");
    } else if (rank == 1) {
        // Processo 1 recebe mensagem
        MPI_Recv(mensagem, 100, MPI_CHAR, 0, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
        printf("Processo 1 recebeu: %s\n", mensagem);
    }
    MPI_Finalize();
    return 0;
}
```

MPI_Send

int MPI_Send(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino, int etiq, MPI_Comm com)

- **mensagem** : endereço inicial dos dados a serem enviados.
 - **cont**: quantidade de dados a serem enviados.
 - **tipo_mpi** : o tipo MPI dos dados a serem enviados:
 - MPI_CHAR, MPI_INT, MPI_FLOAT, MPI_BYTE, MPI_LONG, MPI_UNSIGNED_CHAR, etc.
 - **destino** : ranque do processo destino.
 - **etiq**: etiqueta da mensagem.
 - **com**: comunicador utilizado.
-

MPI_Recv

int MPI_Recv(void* mensagem, int cont, MPI_Datatype tipo_mpi, int origem, int etiq, MPI_Comm com, MPI_Status* estado)

- **mensagem** : endereço inicial do vetor de recepção
- **cont**: número máximo de dados a serem recebidos
- **tipo_mpi** : o tipo MPI dos dados a serem enviados:
 - MPI_CHAR, MPI_INT, MPI_FLOAT, MPI_BYTE, MPI_LONG, MPI_UNSIGNED_CHAR, etc.
- **origem** : ranque do processo origem (* = MPI_ANY_SOURCE).
- **etiq**: etiqueta da mensagem (* = MPI_ANY_TAG).
- **com**: comunicador utilizado
- **estado** : estrutura com três elementos: MPI_SOURCE, MPI_TAG, MPI_ERROR.
 - Origem for MPI_ANY_SOURCE, o elemento MPI_SOURCE indica o ranque do
 - processo emissor da mensagem. Mesma lógica para TAG
 - O elemento MPI_ERROR dessa estrutura contém o código de erro da mensagem recebida

Comunicação Ponto-a-ponto não bloqueantes

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcao\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcão\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```


Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.
- As operações não bloqueantes não retornam imediatamente

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcao\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.
- As operações não bloqueantes não retornam imediatamente
- **MPI_Request** pode ser usado para testar se a operação foi finalizada.
- **MPI_Test** testa se a operação foi concluída
- **MPI_Wait** espera pela chegada real da mensagem.

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcão\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```


Comunicação Ponto-a-ponto não bloqueantes

- **MPI_Isend** : envia um mensagem
- **MPI_Irecv** : recebe uma mensagem.
- As operações não bloqueantes não retornam imediatamente
- **MPI_Request** pode ser usado para testar se a operação foi finalizada.
- **MPI_Test** testa se a operação foi concluída
- **MPI_Wait** espera pela chegada real da mensagem.

```
int main(int argc, char *argv[]) {
    int rank, size;
    MPI_Request req_send, req_recv;
    MPI_Status status;
    // ...
    if (rank == 0) {
        strcpy(msg, "Mensagem nao bloqueante");
        MPI_Isend(msg, strlen(msg) + 1, MPI_CHAR, 1, 0, MPI_COMM_WORLD, &req_send);
        printf("Processo 0 iniciou envio\n");
        // Simula computação enquanto o envio ocorre
        for (int i = 0; i < 3; i++) {
            printf("Processo 0 computando... %d\n", i);
            sleep(1);
        }
        // Garante que terminou
        MPI_Wait(&req_send, &status);
        printf("Processo 0 terminou envio\n");
    } else if (rank == 1) {
        MPI_Irecv(msg, 50, MPI_CHAR, 0, 0, MPI_COMM_WORLD, &req_recv);
        printf("Processo 1 iniciou recepcao\n");
        // Testa continuamente se a mensagem chegou
        while (!flag) {
            MPI_Test(&req_recv, &flag, &status);
            if (!flag) {
                printf("Processo 1 ainda esperando...\n");
                sleep(1); // simula trabalho
            }
        }
        printf("Processo 1 recebeu: %s\n", msg);
        // (opcional aqui, pois já completou)
        MPI_Wait(&req_recv, &status);
    }
    MPI_Finalize();
    return 0;
}
```

MPI_Isend e MPI_Irecv

```
int MPI_Isend(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino, int etiq, MPI_Comm com, MPI_Request *pedido)
```

```
int MPI_Irecv(void* mensagem, int cont, MPI_Datatype tipo_mpi, int origem, int etiq, MPI_Comm com, MPI_Request *pedido)
```

- **pedido**: retorno imediato da função, um manipulador do tipo MPI_Request. Esse manipulador pode ser testado uma única vez (MPI_Test) e retornar, ou o processo pode car em espera ocupada (MPI_Wait) aguardando pela chegada real da mensagem.
-

Buffer

Normalmente, uma área de dados do sistema é reservada para armazenar dados em trânsito. Esse espaço do buffer do sistema é:

- **Invisível para o programador e gerido inteiramente pela biblioteca MPI** ;
- Um recurso **finito** que pode ser esgotado facilmente;
- Muitas vezes misterioso e não bem documentado;
- Capaz de existir no lado de envio, no lado de recebimento, ou em ambos;
- Algo que pode melhorar o desempenho do programa, **pois permite que as operações de envio e recepção possam ser assíncronas.**

Modos de Comunicação MPI

- Síncrono (o mais seguro)
- Pronto (menor sobrecarga para o sistema)
- Bufferizado (desacopla o emissor do receptor)
- Padrão

Modo Padrão

- MPI_Send e MPI_Recv
- Nesse modo, as operações de envio e recepção são bloqueantes, **porém a implementação do MPI decide, em função do tamanho da mensagens, se elas serão enviadas com uso de buffers ou transmitidas no modo síncrono.**
- Uma rotina de envio no modo padrão pode ser iniciada havendo ou não uma rotina de recepção correspondente postada.

Modo Síncrono

- No modo síncrono a rotina de envio pode ser iniciada havendo ou não uma rotina de recepção correspondente já postada.
- Envio completará com sucesso apenas quando uma recepção correspondente for postada e a recepção da mensagem enviada pela rotina de **envio síncrona** for iniciada pela operação de recepção.
- Esse modo **não requer o uso de buffers** no sistema.
- **Podemos assegurar que um programa está seguro se ele executar corretamente utilizando apenas rotinas de envio no modo síncrono.**

MPI_Ssend

```
int MPI_Ssend(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino, int etiq, MPI_Comm com)
```

Modo Pronto

- Esse modo assume que o recv correspondente já foi iniciado (MPI_Recv)
 - **O envio só pode ser iniciado se houver uma rotina de recepção correspondente já postada**
- O MPI evita verificações e sincronizações extras porque o programador garante que o recv já está pronto.
- Isso pode reduzir overhead e melhorar desempenho
- Não usa buffer do usuário;
- Pode ser mais rápido;
- **Requer sincronização correta** ;
- **Uso incorreto gera comportamento indefinido** .

MPI_Rsend

```
int MPI_Rsend(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino, int etiq, MPI_Comm com)
```

Modo Bufferizado

- Operação de envio pode ser iniciada havendo ou não uma operação de recepção correspondente iniciada.
- A operação de envio pode terminar antes de uma recepção correspondente tenha sido postada.
- O uso do modo “bufferizado” permite que a comunicação seja assíncrona e **garante que sempre haverá espaço suficiente para armazenar as mensagens, já que esse espaço é provido pela própria aplicação**
- É função do usuário, e não do sistema, gerenciar a alocação dos buffers

MPI_Bsend

```
int MPI_Bsend(void* mensagem, int cont, MPI_Datatype tipo_mpi, int destino, int etiq, MPI_Comm com)  
int MPI_Buffer_attach(void *buffer, int tam_buffer)
```

PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

Tarefa 14:

Implemente quatro programas MPI com exatamente dois processos. O processo 0 deve enviar uma mensagem ao processo 1, que imediatamente responde com a mesma mensagem.

Meça o tempo total de execução de múltiplas trocas consecutivas dessa mensagem, utilizando `MPI_Wtime`.

Registre os tempos para diferentes tamanhos, desde mensagens pequenas (como 8 bytes) até mensagens maiores (como 1MB ou mais).

Analise graficamente **o tempo em função do tamanho da mensagem** e identifique os regimes onde a latência domina e onde a largura de banda se torna o fator principal. Explique cada implementação e o que cada função faz.

Funções a serem usadas em cada versão: `MPI_Send`, `MPI_Bsend`, `MPI_Rsend` e `MPI_Ssend`

Referências

